

Mini Lesson: Aggregate Methods (CodeGrade)

 <https://github.com/learn-co-curriculum/p3-aggregate-methods>  <https://github.com/learn-co-curriculum/p3-aggregate-methods/issues/new>

Learning Goals

- Write aggregate methods to collect data about objects using their related objects.

Key Vocab

- **Class**: a bundle of data and functionality. Can be copied and modified to accomplish a wide variety of programming tasks.
- **Object**: the more common name for an instance. The two can usually be used interchangeably.
- **Object-Oriented Programming**: programming that is oriented around data (made mobile and changeable in **objects**) rather than functionality. Python is an object-oriented programming language.
- **Function**: a series of steps that create, transform, and move data.
- **Method**: a function that is defined inside of a class.

Introduction

Aggregate methods are useful because they allow you to perform calculations and gather information about the relationships between objects in an efficient manner. In this lesson we will use the example of `Student`, `Enrollment`, and `Course` from the many-to-many lesson to demonstrate some aggregate methods. You can find the file with the classes in the `./lib` directory. The classes have been included here in case you want to play with aggregate methods of your own.

Aggregate methods

Lets try an easy example of an aggregate method that counts the number of courses a `Student` is a part of. For this method all we need to do is look at the count of all the enrollments in the `Student` object.

```
def course_count(self):  
    return len(self._enrollments)
```

Now lets try a more complex example.

Lets say we wanted to add an aggregate method to the `Enrollment` class to figure out how many enrollments were done for any day students were enrolled. Lets make this method a class method.

```
@classmethod  
def aggregate_enrollments_per_day(cls):  
    enrollment_count = {}  
    for enrollment in cls.all:  
        date = enrollment.get_enrollment_date().date()  
        enrollment_count[date] = enrollment_count.get(date, 0) + 1  
    return enrollment_count
```

In this method we iterate through all the enrollments and create a `Dictionary` where the key is the date and the value is the count of enrollments on that date. We increment the count every time we see an enrollment on the same date.

Now lets assume the `Student` has a `grades` attribute. This attribute can be a dictionary where the key is the enrollment and the value is the grade. Lets write a method that would give us the average grade for a student across all the courses they are enrolled in.

```
def aggregate_average_grade(self):  
    # lets assume the grades are stored in a protected attribute called _grades.  
    total_grades = sum(self._grades.values())  
    num_courses = len(self._grades)  
    average_grade = total_grades / num_courses
```

```
return average_grade
```

Since in this example we don't care about the course and only the grades lets get the values from the `grades` dictionary using `self._grades.values()` . We can now sum the grades and divide it by the number of the courses we have which will end up being the average of the students grades across all the enrollments.

Conclusion

Aggregate methods are tools that can help you process data in many-to-many relationships. These methods allow you to compute various aggregate values that can provide insights into the relationships between different entities, such as the number of courses a student is enrolled in, the number of students in a course, or the average grade of a student. They can help you identify trends, patterns, and relationships between different entities.

Resources

- [Python classes](https://docs.python.org/3/tutorial/classes.html)  [_\(https://docs.python.org/3/tutorial/classes.html\)](https://docs.python.org/3/tutorial/classes.html)

This tool needs to be loaded in a new browser window

Load Mini Lesson: Aggregate Methods (CodeGrade) in a new window